Mechanistic Through Lines in ARC Solvers: Iterative Computation, Inductive Bias, and Test-Time Search

@notcomplex_

March 2026

Abstract

Four recent ARC-solving architectures—Universal Reasoning Model (URM) [1], Vision ARC (VARC) [2], Tiny Recursive Model (TRM) [3], and McGovern's test-time adaptation study [4]—differ in apparent motivation yet converge on the same underlying computational strategy: *iterate a hypothesis, score it against demonstrations, select among candidates, and squeeze the search space with strong inductive bias.* This paper synthesizes those four works into a single mechanistic account, identifies the load-bearing contributions that explain performance, and surfaces the deeper open question that none of the papers fully answers: *what is the loop actually computing?*

1 What ARC Actually Demands

The Abstraction and Reasoning Corpus (ARC) was designed as a testbed for *fluid intelligence*: the ability to identify and apply a rule from only a handful of input-output examples, in a domain never encountered before [5]. The benchmark's structure is deliberately hostile to standard machine learning: there are roughly 400 training tasks, each presented as two to four demonstration pairs and one held-out test input, where the test target is unknown. No task appears in two splits. ARC-AGI-2 tightens that screws further by calibrating tasks to be harder for symbolic brute-force while remaining tractable to humans [6].

Two structural facts about ARC shape everything that follows:

The sample budget is tiny. Two to four examples is not enough to fit even a small neural network from scratch. Any successful system must either (a) carry strong priors capable of extrapolating from near-zero data, or (b) use the demonstrations as an on-line search oracle rather than a training set.

Evaluation is pixel-perfect. A single incorrect cell anywhere in the output grid fails the task. This makes probabilistic averaging—averaging multiple distinct hypotheses pixel-by-pixel—almost always harmful, and therefore forces systems toward discrete hypothesis selection rather than soft ensemble output.

Together these two constraints define the design space that URM, TRM, and VARC navigate in distinct but compatible ways.

2 The Unifying Loop

Each of the four architectures reviewed here can be described as an instance of the same computational skeleton:

1. **Encode.** Represent the task (demonstrations + query) as a structured embedding or token sequence.
2. **Refine.** Apply a fixed update operator repeatedly: either a shared-weight recurrent block (URM, TRM, HRM) or a gradient update on a task token (VARC).
3. **Decode.** Read off a candidate output.
4. **Select.** Choose among many candidate outputs by consistency with the demonstrations (voting, majority, or program validation).

The loop is not a metaphor. In URM/TRM/HRM it is literally a recurrent unrolled computation; in VARC it is gradient descent applied per-task at inference time. The convergence of these superficially different approaches on the same four-step skeleton is the central observation this paper attempts to explain.

3 Looped Transformers: URM and TRM

3.1 The core claim: recurrence beats depth

The Universal Transformer (UT) [7] replaced a stack of distinct layers with repeated application of a single shared-weight block. This is architecturally embarrassingly simple, but URM's ablations provide the clearest evidence in the literature that the simplicity is doing real work [1].

URM's headline comparison: at an equal 32× compute multiplier, reallocating computation from non-shared to recurrent passes improves pass@1 on ARC-AGI-1 from 23.75% to 40.0%. More importantly, simply scaling a vanilla (non-recurrent) Transformer's depth and width under the same budget yields only diminishing returns or degradation. This is strong evidence for the hypothesis that *iterative refinement is a better use of compute than additional distinct parameters for this class of task.*

Why might that be? The intuition is that ARC tasks often require a sequence of logically dependent sub-steps—detect a repeating motif, identify which axis of symmetry applies, infer the color mapping, predict the output grid. A fixed-depth feedforward network must commit to a single parse of the task in one pass. A looped model can "circle back": an early pass produces a rough hypothesis; later passes refine, correct, and fill in detail, with the weight tying ensuring that the refinement operator remains consistent across steps. The result is effective depth without parameter explosion.

3.2 What the ablations actually isolate

URM's ablation study goes further than any prior work in this family by decomposing the gain into attributable components [1]:

Nonlinearity is load-bearing, not decorative. Removing or weakening nonlinear components produces monotonic degradation. Replacing SwiGLU with SiLU or ReLU drops accuracy substantially; removing the attention softmax collapses performance to near-random (∼2% pass@1 in their table). This implicates the MLP block—specifically its gating nonlinearity—as the primary source

of representational capacity for ARC-style abstract logic, not the attention head itself. Attention may be doing "routing" while the MLP is doing "thinking."

ConvSwiGLU: convolution in the gate. URM inserts a short depthwise convolution into the MLP gating pathway. Their diagnostic is that placing the convolution *after* expansion is most effective, consistent with the interpretation that it is augmenting the expressive capacity of the nonlinear mixing stage rather than preprocessing features before gating. The practical gain is meaningful: this single addition substantially closes the gap to VARC on ARC-AGI-1.

Truncated Backpropagation Through Loops (TBPTL). Long unrolled recurrences are hard to train: gradients either vanish or explode as they flow backward through many identical blocks. URM's solution—compute gradients only through the later loops, analogous to truncated BPTT in sequence modeling—yields a reported "sweet spot" at skipping the first two inner loops. This is mechanistically interesting: the claim is that early loops are the exploratory phase (rough hypothesis generation) and later loops are the refinement phase (precision correction), and that useful gradient signal lives mostly in the latter.

3.3 TRM: the scratchpad interpretation

TRM arrives at a cleaner mechanistic story for what the latent state *is* [3]. Rather than two networks operating at different biological timescales (as HRM posited), TRM proposes a single tiny 2-layer network and two latent variables with explicit roles: one latent is the current candidate answer, the other is a reasoning scratchpad "acting similarly as a chain-of-thought, except in latent space."

This reframing matters because it gives us an interpretable decomposition: the model is simultaneously maintaining a guess and a process for improving that guess. The scratchpad is not decoded; it is not supervised directly; it exists purely to make the next refinement step more accurate. This is functionally equivalent to hidden chain-of-thought, and the evidence that it works ($\sim$45% pass@1 on ARC-AGI-1 with only 7M parameters) suggests that latent scratchpads may be a computationally efficient substitute for explicit verbal reasoning traces.

TRM also makes test-time augmentation a central method rather than a post-hoc trick: at inference the model runs across $\sim$1000 augmented views (color permutations, dihedral transforms, translations) and reports the plurality answer. This is not a minor implementation detail—it is functionally a test-time search over a structured space of equivalent task restatements, and it contributes substantially to the reported accuracy.

4 Vision as Inductive Prior: VARC

VARC's central argument is that the dominant difficulty in ARC is not computational but representational: the wrong modality creates an artificially hard inductive-bias mismatch [2].

4.1 ARC tasks are visual at their core

Many ARC transformations—reflection, symmetry, gravity, object translation—are primitives of 2D visual and physical experience. They are "difficult" for language models partly because language models tokenize grids into flat sequences, discarding the 2D structure that makes the rule legible to a human. VARC's insight is that framing each task as image-to-image translation with a *canvas* representation (a larger spatial context in which the grids are embedded) allows standard vision architectures—ViTs, U-Nets—to apply 2D inductive priors directly.

The ablation evidence for this claim is unusually sharp: replacing 2D rotary positional embeddings with 1D RoPE drops accuracy from 54.5% to 51.0% (a 3.5 point degradation). That is a large effect for a single hyperparameter change, and it confirms that spatial position structure is contributing real signal, not just acting as a cosmetic prior.

4.2 Test-time training as task adaptation

VARC has no recurrence within the inference pass. Once the model is trained, inference for a new task runs as follows: initialize a task-specific token randomly, fine-tune it (and potentially the entire model) on the few demonstration pairs using gradient descent, then do a feedforward pass to predict the output. This is test-time training (TTT), and it substitutes for the recurrence loop: instead of updating a hidden state by repeatedly applying the same block, VARC updates the model's parameters (or a task projection) by repeatedly applying gradient descent to the demonstration loss.

The quantitative payoff is large:

- Single-view, no TTT: approximately 35.9% pass@1
- Multi-view with TTT: 49.8% pass@1
- Multi-view TTT, pass@2: 54.5%
- Ensemble (ViT + U-Net, both with TTT): 60.4%

The gap between the first and last row—24.5 percentage points—is a direct measure of how much "search" (over views and model states) is doing, compared to the base model capacity alone.

4.3 Multi-view inference as hypothesis marginalization

VARC's multi-view strategy (up to 510 augmented views, majority vote) is structurally identical to TRM's augmentation voting. Both are marginalizing over a transformation group: the group of geometric and color symmetries that should leave the correct answer invariant. This is not just an engineering trick—it is a principled Bayesian operation applied to a finite, discrete group, and the gains it provides suggest that the base models are sensitive to nuisance symmetries that a human solver would automatically factor out.

5 Test-Time Adaptation at Compute Limits: McGovern

McGovern's study [4] is the most practically grounded of the four: it asks whether TRM-style models can be made to work inside the ARC Prize competition's severe compute budget (4×L4 GPUs, 12 hours).

5.1 The compute gap and what it reveals

TRM pretraining requires approximately 750k optimizer steps at a batch size of 768. Competition runtime permits roughly 15k steps at batch size 384. That is a 1/32 compute ratio—a gap that cannot be closed by algorithmic tricks alone.

McGovern's answer is to separate pretraining (unconstrained, done offline) from competition-time fine-tuning (constrained). The finding is that a pretrained model fine-tuned for 12,500 steps reaches

6.67% on semi-private tasks, compared to 10% for the fully pretrained model. That is a 33% relative gap—not negligible, but also not catastrophic. The pretrained weights provide a useful initialization.

5.2 The task-embedding hypothesis fails

The most mechanistically informative finding in McGovern's paper is negative: fine-tuning only the task ID embeddings (while holding the trunk frozen) achieves near-zero accuracy on held-out tasks. This falsifies a clean interpretation in which the task embedding encodes the transformation rule and the frozen trunk executes it. Substantial weight updates throughout the network are required for any given unseen task.

This is an important constraint on interpretive stories. The "soft program in the embedding" hypothesis is appealing, but the evidence suggests that the program is distributed across the full network, not concentrated in a dedicated code vector. Test-time adaptation must reach the trunk.

6 What the Loop is Actually Doing: An Open Synthesis

Taking all four papers together, the following picture emerges—though it must be read as a mechanistic hypothesis, not an established fact.

6.1 Two sources of reasoning, partly separable

ARC performance appears to have two partially separable sources:

Inductive bias / representation alignment. The right prior shrinks the hypothesis space dramatically before any computation is done. VARC's 2D positional encodings, canvas representation, and visual pretraining do this work. URM's recurrent inductive bias (depth via iteration rather than parameter count) does a different version of it. Getting the modality and structure right is not a fine-tuning detail—it is half the problem.

Iterative adaptation / test-time search. Given the right prior, each method then does a form of search: recurrence (URM/TRM) updates a hypothesis state; TTT (VARC, McGovern) updates model weights; augmentation voting (all four) marginalizes over nuisance symmetries. The two sources compound: high-quality priors enable shorter, more reliable search; rich search compensates for imperfect priors.

6.2 The loop as latent algorithm emulation

The theoretical framing that best fits the empirical evidence is that looped transformers are emulating iterative algorithms in latent space. A looped k-layer transformer applied L times can approximate a kL-layer feedforward model, but more than that, the parameter sharing across loops induces a structure closer to a *fixed-point iteration* than to a simple deep pipeline. The loop is searching for a latent representation from which the correct output can be read off, using the same update operator at each step because the same type of "correction" is always needed—check for consistency, refine the current hypothesis, propagate constraints.

TRM's scratchpad latent is the clearest expression of this idea: one half of the hidden state is "the guess," the other is "the evidence for correcting the guess." As loops accumulate, the guess converges and the scratchpad empties. Whether this is analogous to human reasoning steps (object

detection → grouping → rule identification → application) or something altogether different is unknown—none of these papers contains mechanistic interpretability at that resolution.

6.3 Scale enters primarily as test-time compute

Across all four works, the gains that look like "scale" are actually test-time compute: more augmentation views, more refinement loops, more fine-tuning steps, more candidates to vote over. The parameter count of the *base model* is surprisingly small (7M for TRM, 18M for VARC). This suggests that ARC, at least in its current form, rewards *compute efficiency under a tight parameter budget* more than raw parameter scale. This could be a genuine signal about the nature of fluid reasoning (iterative computation is more efficient than parametric memorization for rule induction) or it could be an artifact of the dataset's small size making large models overfit.

ARC-AGI-2's harder tasks and the observed gap between ARC-AGI-1 and ARC-AGI-2 scores—URM achieves 53.8% on V1 but only 16.0% on V2 [1]; TRM achieves 45% on V1 but only 8% on V2 [3]—suggests that whatever these systems have learned is real but narrow. The representations generalize within the difficulty and concept distribution of ARC-AGI-1 but partially fail when that distribution shifts. Whether the core mechanisms (recurrence, visual priors, TTT) are sufficient or whether fundamentally new ideas are required for ARC-AGI-2 remains the central open question.

7 What Is Missing

The papers reviewed here share a common pattern of evidence: "remove the component, observe collapse." URM shows this for nonlinearity and recurrence [1]; VARC shows it for 2D positional encoding [2]. This is valuable but leaves deeper questions untouched:

What intermediate representations arise inside the loops? Are there stable attractors in the latent space that correspond to interpretable concepts ("this is a reflection task," "this is a color-mapping task")? None of the papers contains probing experiments or representational geometry analysis that would answer this.

Can augmentation gains be replaced by better priors? The large contribution of multi-view voting (TRM: ∼1000 views; VARC: up to 510 views) raises the question of whether these gains reflect a genuine marginalization over nuisance symmetries or a compensation for a representation that is insufficiently invariant. If the latter, better architecture might render test-time augmentation unnecessary.

How does the test-time adaptation interact with pretraining? McGovern shows that full fine-tuning dominates embedding-only adaptation, but does not decompose which layers change most or what changes about them. Understanding this would directly inform whether larger pre-trained trunks would help or merely shift the locus of adaptation.

Is the loop convergent? TRM and URM both use fixed (hyperparameter- chosen) loop counts. Neither paper presents evidence of a convergence criterion—a condition under which the system knows it has found a good hypothesis and should stop iterating. Adaptive halting (ACT) is available in the UT framework but does not appear in the ARC-specific papers. Whether adaptive depth would help or be necessary for harder tasks is an open empirical question.

8 Conclusion

Four mechanistically distinct ARC solvers—URM, VARC, TRM, and the McGovern TRM adaptation study—converge on a shared computational strategy: *iterate a hypothesis under strong inductive bias, then select among candidates at test time.* The iteration is implemented as recurrent weight-shared depth (URM/TRM), gradient-based task adaptation (VARC/McGovern), and augmentation-based voting (all four). The inductive bias varies by modality (linguistic tokens vs. 2D vision priors) but serves the same function in each case: drastically shrinking the set of plausible hypotheses before search begins.

The strongest mechanistic claim supported by actual ablation evidence is dual: (1) iterative computation via parameter-shared recurrence is a more efficient allocation of FLOP budget than additional distinct parameters for multi-step abstract rule induction; and (2) the nonlinear (gating) component of the update block is load-bearing—removing it collapses performance, implicating "reasoning" in the MLP rather than the attention head.

What these systems are not yet able to do is explain their own reasoning. The latent loops, scratchpads, and task tokens remain opaque. Closing the gap between "the loop converges to the right answer" and "the loop traces steps recognizable as logical inference" is, arguably, the most important next problem in ARC mechanistic research.

References

[1] Zitian Gao, Lynx Chen, Yihao Xiao, He Xing, Ran Tao, Haoming Luo, Joey Zhou, and Bryan Dai. Universal reasoning model. *arXiv preprint arXiv:2512.14693*, 2025.

[2] Keya Hu, Ali Cy, Linlu Qiu, Xiaoman Delores Ding, Runqian Wang, Yeyin Eva Zhu, Jacob Andreas, and Kaiming He. ARC is a vision problem! *arXiv preprint arXiv:2511.14761*, 2025.

[3] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*, 2025.

[4] Ronan McGovern. Test-time adaptation of tiny recursive models. *arXiv preprint arXiv:2511.02886*, 2025.

[5] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

[6] ARC Prize Foundation. ARC-AGI-2 technical report. 2025.

[7] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. *arXiv preprint arXiv:1807.03819*, 2019.